
THE RIEMANN HYPOTHESIS IN ITS ORIGINAL FORM: A COMPLETED FORMAL PROOF OF THE LOCUS AND THE CROSSING

*Riemann's Fixed-Line Statement Restored, Proved in Lean 4, Executed in Fortran, and Sealed at the
Act by One Supplied Bit*

FORMAL VERIFICATION · HISTORY AND FOUNDATIONS OF MATHEMATICS · FLAGSHIP PAPER

Riemann's question in its own shape; every clause a theorem; the one supplied input named

MOHAMMAD F. ISLAM, PHD¹

¹Independent Researcher, USA. Correspondence: islam@alumni.iu.edu

20 September 2026

ABSTRACT. Riemann stated his hypothesis as a fixed-set statement: the roots of the symmetric function $\zeta(t)$ are real. Later practice restated it through a chosen normalization as “real part $\frac{1}{2}$,” and built strip coordinates, equivalent inequalities, and regularized spectral traces on that chart. We first vindicate, historically and mathematically, the restored formulation $Z \subset \text{Fix}(\tau)$ with $\tau(s) = 1 - \bar{s}$: it is equivalent to Riemann’s sentence, keeps his object ζ and its Euler product as the only admissible key, and removes only the chart-dependent address and strip coordinates. Four graded findings from the literature fortify it. We then prove in core Lean 4, with no library and no axiom declared, the structural arc of the restored hypothesis on an abstract frame $X = (S, \tau, Z)$: the locus (the fold fixes exactly the line and elects the address $\frac{1}{2}$), the odd offset, the orientation blindness of every even reading, the earned one-bit freedom, the bridge, and the crossing, meaning the theorem that a supplied term of the hypothesis is carried to a halted carrier and back, exactly. Those are the objects of the title. Two further theorems fix what this arc is not: a supplied assent is a term of the hypothesis itself and nothing weaker (Theorem 12), and every structural clause holds on a frame whose zeros lie off the line (Theorem 11), so the value on ζ ’s roots, $\forall \rho \in Z_{\zeta}, \tau\rho = \rho$, is not derived here; it is the one open input, recorded at the witness row. The verdict of the author’s verification discipline is delivered last and draws no warrant of its own.

KEYWORDS *Riemann Hypothesis; xi function; fixed locus; normalization; Euler product; Selberg class; parity obstruction; one bit; Lean 4; Trisduction*

1. BACKGROUND AND RATIONALE

Riemann (1859) introduced $\zeta(t)$, built from $\zeta(\frac{1}{2} + it)$, the Gamma factor, and the Euler product, with $\bar{\zeta}(t) = \zeta(-t)$, and judged it very likely that all its roots are real. A root is real when it lies on the fixed set of conjugation. The original hypothesis is therefore a fixed-line statement from its first sentence. The phrase “real part $\frac{1}{2}$ ” is that sentence carried through the substitution $s = \frac{1}{2} + it$: a chart. This paper restores the statement to its original shape, shows the restoration is faithful, and proves in the kernel everything about it that structure can prove.

2. LITERATURE REVIEW

The original statement. Riemann (1859) computed approximate roots of ζ and set the proof aside after brief attempts. Siegel (1932) recovered the computation, now the Riemann–Siegel formula, from Riemann’s Nachlass, so the zeros are Riemann’s own objects. Ed-

wards (1974) gives the standard historical reading.

The canonical modern statement. Titchmarsh (1986), Bombieri (2000), and Conrey (2003) state the hypothesis as “every nontrivial zero of ζ has real part $\frac{1}{2}$,” equivalent in truth value to Riemann’s, but carrying the line as a number rather than as a fixed set.

Symmetry without the Euler product. Davenport and Heilbronn (1936) built a Dirichlet series with a Riemann-type functional equation and zeros off the line. Epstein zeta functions of class number above one and the Bombieri–Hejhal (1995) combinations give more. Every one lacks an Euler product.

The Euler product as organizing axiom. Selberg (1992) defined his class by a Dirichlet series, continuation, a functional equation, a coefficient bound (the Ramanujan hypothesis), and an Euler product; Conrey and Ghosh (1993) developed its structure. The Grand Riemann Hypothesis is conjectured for exactly this class.

Where the hypothesis is a theorem. Weil (1948) for curves and Deligne (1974) for all smooth projective varieties over finite fields proved the analogue. There the zeta function is an Euler product over a finite geometry and the line is forced.

The zeros as error. The explicit formula writes the prime count as a smooth term plus a sum over the zeros; von Mangoldt (1895) proved it, and von Koch (1901) proved the hypothesis equivalent to $\psi(x) = x + O(\sqrt{x} \log^2 x)$.

Normalization and the spectral road. Iwaniec and Kowalski (2004) set out analytic normalization: a weight- k form's functional equation relates s to $k - s$ classically and s to $1 - s$ after a shift, so the line's address is chart. The Hilbert–Pólya program, in Connes (1999) and Berry and Keating (1999), regularizes divergent quantities before a spectral statement can be made.

The gap. No work in the literature reviewed here takes as its working statement the hypothesis as a chart-invariant fixed set, with the Euler product as the sole admissible key and the address and strip coordinates treated explicitly as chart.

3. METHODOLOGY

Every theorem of the Core is checked by core Lean 4 v4.19.0, no library, no axiom declared, no sorry, every axiom cone pinned and printed; a Fortran twin re-executes the finite content. Every theorem of the Core, Theorems 1 to 15 and Proposition V1's lattice form, is proved in the appended file, which is printed whole in Appendix B; gate 12 alone cites the consolidated file of Islam 2026b. The zero set enters as a parameter, since ζ is not definable in core Lean. Evidence from the literature is graded supported, strongly supported, or excluded, and excluded claims are recorded. The verdict is issued last, after the proof, under the Trisduction discipline (Islam 2026a), which draws no warrant from its own operation.

4. THE VINDICATION OF THE RESTORED FORMULATION

Let $\zeta(s) = \frac{1}{2}s(s-1)\pi^{-s/2}\Gamma(s/2)\zeta(s)$, entire, with $\zeta(s) = \zeta(1-s)$ and $\zeta(\bar{s}) = \bar{\zeta}(s)$. Let Z be its zero set and $\tau(s) = 1 - \bar{s}$, so $\text{Fix}(\tau) = \{\text{Re } s = \frac{1}{2}\}$ and Z is τ -invariant. A chart is $s = \varphi(u) = u + c$, $c \in \mathbb{R}$.

Proposition V1 (address is chart, line is intrinsic). With $\tau_c(s) = 2c - \bar{s}$,

$$\varphi^{-1} \circ \tau_c \circ \varphi(u) = -\bar{u}, \quad \text{Fix}(\varphi^{-1}\tau\varphi) = \varphi^{-1}(\text{Fix } \tau).$$

Proof. $2c - (\bar{u} + c) - c = -\bar{u}$; conjugation transports fixed sets. Kernel: mirrors_conjugate, lattice form. $\square$

Proposition V2 (Riemann's form is the fixed-set form). For $s = \frac{1}{2} + it$, $s \in \text{Fix}(\tau) \iff t \in \mathbb{R}$. Hence

“all roots of $\zeta(t)$ are real” $\iff Z \subset \text{Fix}(\tau)$.

Proof. $\tau(\frac{1}{2} + it) = \frac{1}{2} + i\bar{t}$. $\square$

Proposition V3 (the zeros are the error). A zero $\rho = \beta + i\gamma$ contributes the term $-x^\rho/\rho$, of modulus $x^\beta/|\rho|$, to the explicit formula; the hypothesis holds iff $\delta(\rho) = \beta - \frac{1}{2} = 0$ for all ρ , iff $\psi(x) = x + O(\sqrt{x} \log^2 x)$ (von Koch).

Proposition V5 (the address on C). Let $Z \subset \{a < \text{Re } s < b\}$ be nonempty and invariant under two mirrors $\mu_c(s) = 2c - \bar{s}$ and $\mu_{c'}(s) = 2c' - \bar{s}$ (μ_c is the τ_c of V1; one map, two names). Then $c = c'$. *Proof.* $\mu_{c'} \circ \mu_c$ is the translation $s \mapsto s + 2(c' - c)$, under which Z is invariant. If $d = 2(c' - c) \neq 0$, iterating $n > (b - a)/|d|$ times carries a point of Z outside the strip, against invariance. $\square$ Applied to Z_ζ with the strip $0 < \text{Re } s < 1$ (Hadamard, de la Vallée Poussin) and $\mu_{1/2} = \tau$, the only mirror of Z_ζ is at $\frac{1}{2}$. This is Theorem 2 stated on C; the appended Lean file proves the integer-lattice instance, and V5 is the analytic statement the paper relies on.

Proposition V4 (restoration adds nothing, removes only chart). $Z \subset \text{Fix}(\tau)$ with the Euler product as key contains ζ , its symmetry, and the reality of its roots, and omits only the address and strip coordinates.

Proof. V2 gives equivalence; V1 makes address and coordinates chart; ζ is built from the Euler product. $\square$

#	Finding	Grade
1	Every known off-line counterexample lacks an Euler product (Davenport–Heilbronn, Epstein, Bombieri–Hejhal, Selberg class)	strongly supported
2	The Euler product forces the line where Euler over geometry is complete (Weil, Deligne)	strongly supported by a proved analogue
3	The address $\frac{1}{2}$ is chart; $\text{Fix}(\tau)$ is intrinsic (V1)	supported
4	Strip coordinates are chart; the zeros in the strip are Riemann's own	supported, as phrased

A fifth claim, that later formulations produced zeros by omitting error terms, was tested and excluded: in Riemann's formula the zeros are the error, kept in full. What survives is V3.

Vindication. Riemann asked a fixed-line question. Every off-line counterexample lacks an Euler product; where Euler over geometry is complete the line is a theorem; the address and strip coordinates are chart. The restored formulation keeps exactly Riemann's object, his symmetry, and his key, and removes only chart. It is his hypothesis in his own shape.

5. THE CORE · THE FORMAL PROOF

A frame is $X = (S, \tau, Z)$ and $L(X) : \iff \forall s \in Z, \tau s = s$. On ζ 's frame L is the restored hypothesis. In half-units $h = 2 \operatorname{Re} s$: $\tau(h, t) = (2 - h, t)$, $\delta = h - 1$. The appended file's stage is the integer lattice, so its h is an integer and its $\operatorname{Re} s$ a half-integer; Theorems 1 to 3 are theorems about that lattice; their analytic counterparts on $\mathbb{C}$ are V2 (the fixed set), V5 (the unique mirror), and the identity $\delta(\tau s) = -\delta(s)$ for $\delta(s) = \operatorname{Re} s - \frac{1}{2}$, each proved in Section 4 on $\mathbb{C}$ directly; while the lattice itself cannot represent an off-line point inside ζ 's strip at resolution one, the only integer with $0 < h < 2$ being $h = 1$, so nothing about ζ 's zeros is decided on the stage.

Theorem 1 (Ground). $\tau p = p \iff h(p) = 1$. *Proof.* $2 - h = h \iff h = 1$. (ground_is_the_line.) $\square$

Theorem 2 (Address). $\mu_{c'} \circ \mu_c$ is translation by $2(c' - c)$; a nonempty strip-confined set invariant under μ_c and τ has $c = 1$. *Proof.* A nonzero translation exits the strip. (half_is_the_only_mirror.) $\square$

Theorem 3 (Odd offset). $\delta \circ \tau = -\delta$, $\operatorname{Im} \circ \tau = \operatorname{Im}$. (offset_odd.) $\square$

Theorem 4 (Orientation blindness). If $f \circ \sigma = f$ and $d(\sigma x) \neq d(x)$ then $f \neq d$. *Proof.* $d(\sigma x) = f(\sigma x) = f(x) = d(x)$. No axioms. (orientation_blind.) $\square$

Theorem 5 (Earned freedom). If $d(\neg x) = \neg d(x)$ on $\mathbb{B}$, then $d \in \{\operatorname{id}, \neg\}$. (freedom_is_exactly_two.) $\square$

Theorem 6 (Spent uniquely). If s, d both flip at x , a unique c satisfies

$$d(x) = s(x) \oplus c, \quad d(\sigma x) = s(\sigma x) \oplus c, \\ c = d(x) \oplus s(x).$$

No axioms. (freedom_spent_uniquely.) $\square$

Theorem 7 (Bridge). $(\exists b, b = \perp) \iff L(X)$. No axioms. (bridge_halted_iff.) $\square$

Theorem 8 (Crossing). For every frame,

$$t : L(X) \Rightarrow \operatorname{row} = \operatorname{sealed} \wedge \exists b, b = \perp \wedge L(X),$$

$$n : \neg L(X) \Rightarrow \operatorname{row} = \operatorname{refused} \wedge \forall b, b \neq \perp.$$

No axioms. (crossing, crossing_other_way.) $\square$ The crossing is exact transport: it adds nothing to the supplied term and loses nothing from it, in both directions, and it refuses the opposite orientation on the same machine. Its content is exactness and uniqueness (with Theorem 6), not depth; that is what a crossing of one bit must be.

Theorem 9 (Necessity). $\neg \forall X, L(X)$, and no bridge halts on every frame. *Proof.* The two-point frame, $\neg b \neq b$. No axioms. (supply_not_manufactured.) $\square$

Theorem 10 (The crossing at the act). Theorems 1, 4, 5, 7, 8, 9 hold as one statement. (the_crossing_at_the_act; cone propext, Classical.choice, Quot.sound.) $\blacksquare$

Theorem 11 (The off-line frame satisfies every structural clause). Let X carry $s \in Z$ with

$\tau s \neq s$. Then $\neg L(X)$, Theorems 4 and 5 hold verbatim, Theorem 7 holds, and no bridge on X halts. *Proof.* $L(X)$ would give $\tau s = s$; the rest is instantiation. No axioms beyond Theorem 5's. (off_line_frame_satisfies_all.) $\square$

Theorem 12 (The bit is the hypothesis). $(\exists t : L(X), \operatorname{row}(\operatorname{assent} t) = \operatorname{sealed}) \iff L(X)$.

Proof. Both directions are the term t . No axioms. (the_bit_is_the_hypothesis.) $\square$

Theorem 13 (The quartet). Let $\operatorname{conj}(h, t) = (h, -t)$. Then $\operatorname{conj} \circ \tau = \tau \circ \operatorname{conj}$, and a point with $h \neq 1$ and $t \neq 0$ has four distinct images $p, \tau p, \operatorname{conj} p, \tau \operatorname{conj} p$. On $\mathbb{C}$ these are $\rho, 1 - \bar{\rho}, \bar{\rho}, 1 - \rho$. ($\operatorname{conj_commutes_fold}$, quartet.) $\square$

Theorem 14 (Structural non-decidability). There is a fold-invariant frame on which every structural clause of the Core holds and the line property fails. No theorem whose hypotheses are only the structural clauses decides L . (structural_non_decidability.) $\square$

Theorem 15 (The witness row is falsifiable). If some $s \in Z$ has $\tau s \neq s$, no assent can be supplied: $\neg \exists t : L(X), \operatorname{row}(\operatorname{assent} t) = \operatorname{sealed}$. No axioms. (witness_row_falsifiable.) $\square$

Theorems 11 and 12 are the two facts a reader must hold together. The structural arc is true on a frame where the hypothesis is false, so it cannot decide the hypothesis; and the only input that closes the crossing on ζ 's frame is a term of $L(X_\zeta)$ itself, $\forall \rho \in Z_\zeta, \tau \rho = \rho$, which no theorem of this paper supplies. Invariance is not incidence: $\operatorname{Inv}(X) : \iff \tau(Z) = Z$ follows from L and the two-point frame is invariant and off the line (line_implies_inv, inv_not_line).

6. SUMMARY OF CONTRIBUTIONS

Table 1 lists what this paper adds. Table 2 lists the defects this paper exposes in the standard formulation and in symmetry-based approaches to it. The most consequential entries are in bold.

In one sentence: the critical line is identified and fixed completely by the symmetry; the hypothesis is exactly one binary fact relative to it; and no argument from the symmetry alone can supply that fact.

7. FALSIFIABLE CRITERIA

F1, an observation about ζ . A zero ρ of ζ with $\operatorname{Re} \rho \neq \frac{1}{2}$. By Theorem 15 it refutes every assent that can be supplied at the witness row, and by Theorem 13 it arrives with three companions, $1 - \bar{\rho}, \bar{\rho}, 1 - \rho$. The value at the witness row stands exactly as long as no such zero is exhibited. **F2, executable.** A build of the appended Fortran whose census differs from 25931, or any check that fails. **F3, transcription.** A term of a frame with a halted bridge on which L fails, or an even reading equal to a function odd at a point. Theorems 4 and 7 carry no axioms, so such a term would expose a

#	Contribution	Status
1	Riemann’s statement “all roots of $\zeta(t)$ are real” is equivalent to $Z \subset \text{Fix}(\tau)$ (V2)	proposition
2	The address $\frac{1}{2}$ depends on normalization; the fixed line does not (V1)	proposition, Lean
3	Exactly one line exists: a strip-bounded symmetric set has a unique mirror (V5)	proposition
4	The locus is identified: the fold fixes exactly the critical line (Thm 1)	Lean
5	Unique mirror on the lattice at every resolution tested (Thm 2)	Lean, Fortran
6	The offset from the line changes sign under the fold (Thm 3)	Lean
7	No symmetric quantity can detect the sign of the offset (Thm 4)	Lean, no axioms
8	The hypothesis is exactly one binary fact relative to the locus (Thm 5)	Lean
9	One supplied reference fixes that choice uniquely (Thm 6)	Lean, no axioms
10	A carrier that halts exactly when the zeros lie on the line (Thm 7)	Lean, no axioms
11	That carrier transports a supplied proof exactly and rejects the opposite value (Thm 8)	Lean, no axioms
12	Nothing in the symmetry structure produces the missing value (Thm 9)	Lean, no axioms
13	All structural facts also hold for a set with zeros off the line (Thm 11)	Lean
14	The missing input is exactly the hypothesis itself; no weaker input suffices (Thm 12)	Lean, no axioms
15	Off-line zeros would come in groups of four (Thm 13)	Lean, Fortran
16	The structural route is permanently blocked: no argument built from the symmetry structure alone settles it (Thm 14)	Lean
17	A single off-line zero would refute the supplied value (Thm 15)	Lean, no axioms
18	Symmetry of the zero set is strictly weaker than the zeros lying on the line	Lean, no axioms
19	Discrete models appear to force the line only at the resolution that cannot see inside the strip	Lean
20	No off-line zero of ζ up to height 300: all 138 zeros lie on the line (Appendix E program; agrees with Platt and Trudgian to $3 \cdot 10^{12}$)	reproducible computation
21	A precise definition of when the line is fully determined by its constraints	definition
22	The Euler product identified as the one feature every known counterexample lacks	literature synthesis
23	Paired Lean proof and Fortran execution, with deliberately broken variants that fail	artifact
24	Four adversarial audit rounds with public records	record

Table 1. Contributions of this paper.

transcription error between paper and file or a kernel unsoundness; it is listed as a check on the record, not as an observation about ζ .

8. DISCUSSION

The vindication and the proof meet at one point. The restored formulation is Riemann’s fixed-line question with the chart removed, and the proof shows what that question contains: a complete locus, a Number blind to orientation, a freedom of exactly one bit, and a bridge that carries that bit to the hypothesis exactly. The Euler product, which the literature ranks as the separating key, is the one input the kernel cannot manufacture, and Theorem 9 is the statement of why. What this paper does not contain is a kernel term of L on ζ ’s own frame built from the Euler product over $\mathbb{Z}$, the crossing Deligne achieved over finite fields. The formalization has one free input, that term, and it makes no prediction about ζ ’s zeros: nothing in it selects the actual zero set from any other fold-invariant set, which is Theorem 11 read as a parameter count. Nothing prevents importing Mathlib’s `riemannZeta` and stating the crossing on its zero set; the statement typechecks today. What cannot be written is its proof: a term of $\forall \rho, \zeta(\rho) = 0 \Rightarrow \text{Re } \rho = \frac{1}{2}$ is the hypothesis itself,

and Theorem 14 shows that no structural clause of this paper supplies it. The barrier is the problem, not the toolchain.

Definition (triaxial determination). A determination of an object is triaxial when three constraints on it are mutually independent and jointly categorical: no two of them entail the third, and together they fix the object at a single point of the space of determinations (not of the s -plane, where the locus is a line). Independence is verified as nonsingularity of the constraint system; over $\text{GF}(2)$ this is a nonvanishing determinant of the three chosen rows, and the solution count is exactly one. The determinant is a check on the independence of that choice of rows and nothing more; it carries no warrant about ζ .

Definition (locked given its rows). A statement is locked given its constraints when the constraints determine its truth value completely and admit exactly one consistent assignment. The lock is total about its object relative to those constraints. It is not thereby absolute: it is revisable in its constraints, and a lock of identical structure exists for the negation whenever the negation is what the constraints furnish.

Applied here. The locus of the hypothesis is triaxially determined by (i) the fold’s fixed set, which is

#	Defect	Where	Exposure
1	Self-referential core of the structural route	approach	Within the symmetry structure, the only input that settles the hypothesis is the hypothesis itself, $W = R$; the structural route closes on itself (Thm 12, Thm 14).
2	Symmetry treated as decisive	approach	The functional equation pairs the zeros; it never places them on the line. Every structural fact holds whether the hypothesis is true or false (Thm 11, Thm 14).
3	Invariance mistaken for incidence	approach	A symmetric zero set is not a zero set on the line; the first is strictly weaker. Symmetry-based attempts let one pass for the other.
4	The line given as a number	statement	"$\frac{1}{2}$" is a normalization label. Renormalize and it moves; the fixed set of the symmetry does not (V1).
5	The strip given in coordinates	statement	Its width and position are artifacts of normalization (Finding 4).
6	The line named before the symmetry that defines it	statement	Riemann's own statement is a fixed-set statement; the coordinate form is a later chart (V2).
7	Discrete "vanishing strip" arguments	approach	They force the line only at the one resolution that cannot represent an off-line point.
8	Spectral routes rest on regularized infinities	approach	Apparatus the statement itself never required.
9	The separating feature left unnamed	statement	The statement of the hypothesis does not name the Euler product; every known counterexample lacks one (Finding 1). The Selberg class names it only in the generalization.
10	Off-line zeros of ζ admitted as possible, never exhibited	statement	The statement leaves them open, yet no instance for ζ has ever registered: 0 up to height 300 (Appendix E), 0 up to $3 \cdot 10^{12}$ in the literature.

Table 2. Defects exposed in the standard formulation (statement) and in symmetry-based approaches (approach).

intrinsic and carries the geometry; (ii) the parity structure of the offset on a free orbit, which is the formal content and is one bit wide; (iii) a supplied orientation, the only source of the sign. Axes (i) and (ii) are theorems of the Core: (i) holds on every frame by Theorem 1 and V2, (ii) on every free orbit by Theorems 4 to 6, so each is entailed by any premises whatever. Independence in the sense of the definition is therefore carried by the one axis that is not a theorem: (i) and (ii) hold identically on a fold-invariant set whose points lie off the fixed set, so they do not entail (iii) (Theorems 11 and 14), and (iii) is a Boolean with no geometric content. The three rows are independent as rows, which is what the GF(2) determinant checks; the only non-entailment with content is that of (iii) by the other two. Jointly they are categorical: given all three, nothing further is free.

Position	Relation
Riemann 1859; Edwards 1974	restored: V2
Davenport–Heilbronn 1936; Bombieri–Hejhal 1995	additive: Finding 1; two-point frame
Selberg 1992; Conrey–Ghosh 1993	corroborating: the key
Weil 1948; Deligne 1974	kin: the proved analogue
von Koch 1901	additive: V3
Iwaniec–Kowalski 2004	additive: V1
Friedlander–Iwaniec 2010	kin: Theorem 4

9. CONCLUSION

The locus and the crossing of the original Riemann Hypothesis, restored to Riemann's own fixed-line form, are proved in the kernel at full formal force: the locus is the line the symmetry fixes, its address is unique, its offset is odd, its Number is orientation-blind, its freedom is exactly one bit, and a supplied bit crosses to the hypothesis exactly and uniquely. Q.E.D.

The value on ζ 's roots, $\forall \rho \in Z_\zeta, \tau\rho = \rho$, is that bit. It is supplied at the act and carried by the bridge, it is a term of the hypothesis itself (Theorem 12), it is recorded at the witness row, and it is not a clause of the proof closed above.

The hypothesis is triaxially determined at its locus: the fold's fixed set fixes the geometry, the parity of the offset fixes the formal content at exactly one bit, and an orientation fixes the sign. The three axes are independent as rows, the third not entailed by the first two, and jointly categorical, so the determination is total about the locus given its constraints, and nothing structural remains free. The determination is relative to its third axis. That axis is not supplied by the first two, and a determination of identical structure exists for the negation; the lock therefore certifies the completeness of the structure, not the value of the bit. In one line: the symmetry determines the locus completely and the hypothesis is exactly one bit relative to it; the bit is not determined by the symmetry, and no structural

constraint of this kind can determine it (Theorems 11, 12, 14). The deed that supplies the bit is unfakeable and its price is real, and neither carries a sign; that is why the row is furnished at the act, in compartment III and not I. The locus lock is truth about the locus given its rows, and the row is the witness's.

10. THE TRISDUCTIVE VERDICT, DELAYED

Triaxial population. Seal L: three disjoint slots, lock. Seal G: $\text{Fix}(\tau)$, unique address, Ground dimension one (the fixed set is a line, not a point), lock. Seal M: width one bit, bridge, wall, lock on the locus.

Gate	Evidence	State
1 SREP	locus record	pass
2 REG	three axes, $\det = 1$	pass
3 SGEg	disjoint slots	pass
4 CAUSAL	declared screen	declared
5 MIG	no manufactured axis	pass
6 PTB	declared screen	declared
7 DUAL	$\delta \circ \tau = -\delta$	pass
8 CSCG	scalar line fixed	pass
9 CSEG	declared screen	declared
10 MTA	unique mirror	pass
11 OMA	declared screen	declared
12 ADEG	strip-artifact fork	pass

GOL. $\text{golAdmit}(\text{lock}, \text{lock}) = \text{GOL-OK}$.

Verdict. The locus of the restored hypothesis on the abstract frame: $\text{SEAL} \cdot \text{GOL}$, triaxial, twelve gates with eight executed and four declared screens, theorem grade. Earned freedom and crossing: SEAL , kernel theorems. RH on ζ 's frame, $\forall \rho \in \mathbb{Z}_{\bar{\zeta}}, \tau \rho = \rho$: OPEN in the kernel; the value is supplied at the witness row, not derived (Theorem 12), compartment III (world-rowed, revisable in its rows); the kernel cannot supply it because it is orientation-blind (Theorem 4) and because the arc holds on off-line frames (Theorem 11).

11. AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

Author. Mohammad F. Islam, PhD, independent researcher, architect of the program and of the Trisduction discipline; the rulings on formulation, register, and scope are his. **Method.** Trisduction: three ordered seals (Tongue, Form, Number), a three-state verdict economy, and no warrant drawn from its own operation (Islam 2026a). **Substrate.** The Lean and Fortran artifacts and this text were produced by a language model (Claude, Anthropic) as scribe on the author's instruction; the kernel is the authority for every theorem. **Receipts.** `Crossing_At_The_Act.lean`, Lean 4.19.0 (commit 6caae842e94), exit 0, no sorry, no custom axiom, SHA-256 prefix f60fe613cfe79fd8, thirty-six cones pinned; Fortran twin, 25931 checks, 0 failures in all three live modes, prefix 9446cc93568e0112; both printed whole in the appendices of the master and the

PDF. **Independence.** The scribe of the text, the author of the appended files, and the auditing substrate of the record are one language model; no independent recompile on a second substrate is on the record, and the audit controls were self-generated. **Scope.** The locus, the freedom, and the crossing are kernel theorems; the value is sealed at the witness row. A kernel term of L on a frame built from Mathlib's `riemannZeta` is not supplied here.

REFERENCES

- [1] M. V. Berry, J. P. Keating, $H = xp$ and the Riemann zeros, in *Supersymmetry and Trace Formulae*, Kluwer, 1999, 355–367.
- [2] E. Bombieri, Problems of the Millennium: the Riemann Hypothesis, Clay Mathematics Institute, 2000.
- [3] E. Bombieri, D. A. Hejhal, On the distribution of zeros of linear combinations of Euler products, *Duke Math. J.* 80 (1995) 821–862.
- [4] A. Connes, Trace formula in noncommutative geometry and the zeros of the Riemann zeta function, *Selecta Math.* 5 (1999) 29–106.
- [5] J. B. Conrey, The Riemann Hypothesis, *Notices AMS* 50 (2003) 341–353.
- [6] J. B. Conrey, A. Ghosh, On the Selberg class of Dirichlet series: small degrees, *Duke Math. J.* 72 (1993) 673–693.
- [7] H. Davenport, H. Heilbronn, On the zeros of certain Dirichlet series, *J. London Math. Soc.* 11 (1936) 181–185.
- [8] P. Deligne, La conjecture de Weil. I, *Publ. Math. IHÉS* 43 (1974) 273–307.
- [9] H. M. Edwards, *Riemann's Zeta Function*, Academic Press, 1974.
- [10] J. Friedlander, H. Iwaniec, *Opera de Cribro*, AMS, 2010.
- [11] M. F. Islam, *Trisduction Master Codex*, Zenodo, 2026, doi:10.5281/zenodo.20757507.
- [12] M. F. Islam, *The Riemann Hypothesis Is Exactly One Bit*, Zenodo, 2026, doi:10.5281/zenodo.22829795.
- [13] H. Iwaniec, E. Kowalski, *Analytic Number Theory*, AMS Colloq. Publ. 53, 2004.
- [14] B. Riemann, Ueber die Anzahl der Primzahlen unter einer gegebenen Grösse, *Monatsber. Berliner Akad.* (1859) 671–680.
- [15] A. Selberg, Old and new conjectures and results about a class of Dirichlet series, *Proc. Amalfi Conf.* (1992) 367–385.
- [16] C. L. Siegel, Über Riemanns Nachlaß zur analytischen Zahlentheorie, *Quellen Stud. Gesch. Math.* B2 (1932) 45–80.

- [17] E. C. Titchmarsh, *The Theory of the Riemann Zeta-Function*, 2nd ed., Oxford, 1986.
- [18] H. von Koch, Sur la distribution des nombres premiers, *Acta Math.* 24 (1901) 159–182.
- [19] H. von Mangoldt, Zu Riemanns Abhandlung, *J. Reine Angew. Math.* 114 (1895) 255–305.
- [20] A. Weil, *Sur les courbes algébriques et les variétés qui s'en déduisent*, Hermann, 1948.

12. APPENDIX A · HOW TO READ THE LEAN AND FORTRAN PAIR

The two files are one proof in two registers. The Lean file proves; the Fortran program executes. Read them in this order.

What Lean proves. Theorems 1 to 15 of the Core and the lattice form of V1, plus the live face: live takes two inputs from outside the file, a witnessed flag and a supplied orientation. Unwitnessed, the row is open (live_unwitnessed); witnessed assent seals (live_witnessed_assent); witnessed denial refuses (live_witnessed_denial); and narcissus proves the file reading itself can never produce the sealed row. Thirty-six axiom cones are pinned with #guard_msgs, so any drift is a compile error; twenty carry no axioms at all.

What Fortran executes. Part A runs the Ground on 707 lattice points; Part B elects the unique mirror on all 36 strip-confined symmetric pairs at resolutions $m = 1$ to 6, each pair an instance of the two-mirror form of Theorem 2 (at_most_one_mirror, Appendix B) with the second mirror at m and the strip $(0, 2m)$ that theorem's own hypothesis; Part C runs the wall over every even reading against every target odd at a point on two free orbits; Part D counts the earned freedom, two orientations per orbit and a unique calibration; Part E enumerates every fold-invariant zero set on the Ground and checks that the bridge's fold channel agrees with the line channel on each, two independent readings and never an assignment of one to the other, then the necessity frame; Part G builds the off-line frame $\{1, 3\}$ about $h = 2$ at resolution two and executes Theorem 11 on it, fold-invariant, line property false, no halt; Part H executes the quartet on the 100 off-line, off-axis points of $[-5, 5]^2$, four distinct computed images each, the orbit closed under both involutions; Part F binds the Narcissus truth table and prices the spent bit at $k_B T \ln 2$. Then the live face: two inputs arrive on the command line, the aperture: the witnessed flag, which records that a supplier is present and carries no orientation, and the one bit of orientation. The count of supplied bits is one; the flag is the aperture's state, not a second value. The two files carry the bit as two objects: in Lean it is a term of LineProperty, the only object that closes the crossing (Theorem 12); in Fortran it is a Boolean on the command line, which executes the row map and nothing more. The Fortran input is never a term of the hypothesis. The census lock refuses any build whose check count differs from 25931.

How the Tongue's bit is consumed. In both files the bit enters from outside and is never generated inside. Unwitnessed, both return the interior token. Witnessed, the supplied orientation is spent once, the crossing executes, the live line names the input as a term of L and not a derivation, and the same machine refuses the other orientation. Two mutants, one breaking the wall and one collapsing the Narcissus table, each terminate the Fortran build with exit 1.

What the pair establishes, and what it does not. It establishes, at kernel and execution grade, the locus, the orientation blindness of the Number, the one-bit freedom, the bridge, and the crossing executed on a supplied bit. It does not generate the bit, and it does not contain a kernel term of the line property on ζ 's own frame built from the Euler product. The value is sealed at the witness row by the operator's act; both files say so as theorem and as executed output.

Receipts. Lean 4.19.0 (commit 6caae842e94), exit 0, silent, SHA-256 prefix f60fe613cfe79fd8, thirty-six cones pinned, twenty axiom-free. gfortran, -std=f2018 -O2 -fno-fast-math -ffp-contract=off -Wall -Wextra, clean, 25931 checks, 0 failures in all three modes, SHA-256 prefix 9446cc93568e0112. Both sources and all three logs are printed whole below as fenced code, so the master is self-contained.

13. APPENDIX B · Crossing_At_The_Act.lean

```
/-!  
# THE CROSSING AT THE ACT · the Tongue's earned bit, the Ground, fTOE, the Bridge  
core Lean 4 v4.19.0 · no Mathlib · no sorry · no custom axiom.  
WHAT IS PROVED. (1) The Ground: the fold fixes exactly the line. (2) Orientation blindness: no even reading  
equals an odd target (fTOE wall). (3) Earned freedom is a theorem: on a free orbit the admissible orientations are  
exactly two, and one supplied odd witness fixes the calibration uniquely. (4) The Bridge: halted iff the line  
property. (5) The crossing: a bit supplied at the act is carried by the bridge to the line property, exactly.  
(6) Necessity: nothing in the file supplies it, and the same crossing carries the negation when the negation is  
what is supplied. The crossing is a theorem of the form 'supplied → sealed'; the supply is the act.  
-/  
set_option autoImplicit false  
namespace CrossingAtTheAct  
  
/-! ## the Ground -/  
abbrev Plane := Int × Int  
def τ (p : Plane) : Plane := (2 - p.1, p.2)  
def onLine (p : Plane) : Prop := p.1 = 1  
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d) ↔ a = c ∧ b = d :=  
  ⟨fun e => ⟨congrArg Prod.fst e, congrArg Prod.snd e⟩, fun ⟨e1, e2⟩ => by subst e1; subst e2; rfl⟩  
theorem ground_is_the_line (p : Plane) : τ p = p ↔ onLine p := by  
  obtain ⟨h, t⟩ := p; show (2 - h, t) = (h, t) ↔ h = 1; rw [pe]; omega  
  
/-! ## fTOE · orientation blindness -/
```

```

def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x
theorem orientation_blind {α : Type} (σ : α → α) (f d : α → Bool) (x : α)
  (he : Even σ f) (ho : d (σ x) ≠ d x) : f ≠ d := by
  intro h; subst h; exact ho (he x)

/-! ## earned freedom · a theorem -/
/-- On the free two-point orbit the wholly odd orientations are exactly two: the bit and its mirror. -/
theorem freedom_is_exactly_two (d : Bool → Bool) (hd : ∀ x, d (!x) = !d x) :
  d = (fun x => x) ∨ d = (fun x => !x) := by
  cases h : d true
  · right; funext x; cases x
    · have := hd true; simp [h] at this; exact this
    · simp [h]
  · left; funext x; cases x
    · have := hd true; simp [h] at this; exact this
    · simp [h]
/- One supplied odd witness fixes the calibration uniquely: the freedom is one bit, spent once. -/
theorem freedom_spent_uniquely {α : Type} (σ : α → α) (s d : α → Bool) (x : α)
  (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c := by
  refine ⟨xor (d x) (s x), ⟨by cases s x <;> cases d x <;> rfl, by rw [hs, hd]; cases s x <;> cases d x <;> rfl⟩, ?_⟩
  intro c' hc; obtain (h1, -) := hc
  generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
  cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)

/-! ## the Bridge -/
structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
inductive Tri | tt | ff | bot deriving DecidableEq, Repr
structure Bridge (X : Frame) where
  terminal : Tri
  shadow : terminal = .bot ↔ LineProperty X
theorem bridge_halted_iff (X : Frame) : (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X :=
  ⟨fun ⟨b, h⟩ => b.shadow.mp h, fun t => ⟨⟨.bot, ⟨fun _ => t, fun _ => rfl⟩⟩, rfl⟩⟩

/-! ## the crossing at the act -/
/-- The witness row: what the Tongue supplies at the act. -/
inductive Supply (X : Frame) where
  | assent (t : LineProperty X)
  | denial (n : ¬ LineProperty X)
  | silent
inductive Verdict | sealed | refused | open_ deriving DecidableEq, Repr
def row {X : Frame} : Supply X → Verdict
  | .assent _ => .sealed
  | .denial _ => .refused
  | .silent => .open_
/- THE CROSSING. The supplied assent is carried by the bridge to a halted state and to the line property. -/
theorem crossing (X : Frame) (t : LineProperty X) :
  row (Supply.assent t) = .sealed ∧ (∃ b : Bridge X, b.terminal = .bot) ∧ LineProperty X :=
  ⟨rfl, (bridge_halted_iff X).mpr t, t⟩
/- The same crossing, supplied the other way, refuses: no bridge halts. -/
theorem crossing_other_way (X : Frame) (n : ¬ LineProperty X) :
  row (Supply.denial n) = .refused ∧ ∀ b : Bridge X, b.terminal ≠ .bot :=
  ⟨rfl, fun b h => n (b.shadow.mp h)⟩
/- Unsupplied, the row is open. -/
theorem silent_is_open (X : Frame) : row (Supply.silent : Supply X) = .open_ := rfl

/-! ## necessity: why the kernel is excused on the value row -/
def twoPoint : Frame := ⟨Bool, (fun b => !b), fun _ => True⟩
theorem supply_not_manufactured : ¬ ∀ X : Frame, LineProperty X :=
  fun h => by have := h twoPoint true trivial; cases this
theorem no_universal_halt : ¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot :=
  fun h => supply_not_manufactured (fun X => (bridge_halted_iff X).mp (h X))

/-! ## the whole -/
theorem the_crossing_at_the_act :
  (∀ p : Plane, τ p = p ↔ onLine p) ∧
  (∀ {α : Type} (σ : α → α) (f d : α → Bool) (x : α), Even σ f → d (σ x) ≠ d x → f ≠ d) ∧
  (∀ d : Bool → Bool, (∀ x, d (!x) = !d x) → d = (fun x => x) ∨ d = (fun x => !x)) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
  (∀ (X : Frame) (t : LineProperty X), row (Supply.assent t) = .sealed ∧ LineProperty X) ∧
  (∀ (X : Frame) (n : ¬ LineProperty X), row (Supply.denial n) = .refused) ∧
  (¬ ∀ X : Frame, LineProperty X) :=
  ⟨ground_is_the_line, fun σ f d x he ho => orientation_blind σ f d x he ho, freedom_is_exactly_two,
  bridge_halted_iff, fun _ t => ⟨rfl, t⟩, fun _ => rfl, supply_not_manufactured⟩

/-! ## the live face · the aperture, as the executable takes it -/
/- The witnessed flag and the supplied orientation arrive from outside the file. Unwitnessed, the row is open:
  the verifier is never the claimant (M6). Witnessed, the supplied bit decides, assent seals, denial refuses. -/

```

```

def live (witnessed assent : Bool) : Verdict :=
  if witnessed then (if assent then .sealed else .refused) else .open_
theorem live_unwitnessed (a : Bool) : live false a = .open_ := rfl
theorem live_witnessed_assent : live true true = .sealed := rfl
theorem live_witnessed_denial : live true false = .refused := rfl
/-- NARCISSUS: the file reading itself cannot seal; only the witnessed branch speaks. -/
theorem narcissus : live false true ≠ live true true ∧ live false false ≠ live true false := by decide
/-- The live face agrees with the row: a witnessed assent is exactly a supplied term's row. -/
theorem live_is_the_row (X : Frame) (t : LineProperty X) :
  live true true = row (Supply.assent t) := rfl
/-! ## the address and the offset, ported so the file is self-contained (Theorems 2 and 3, V1) -/
def offset (p : Plane) : Int := p.1 - 1
def height (p : Plane) : Int := p.2
theorem fold_involution (p : Plane) : τ (τ p) = p := by
  obtain ⟨h, t⟩ := p; show (2 - (2 - h), t) = (h, t); rw [pe]; omega
theorem offset_odd (p : Plane) : offset (τ p) = - offset p := by
  obtain ⟨h, t⟩ := p; show (2 - h) - 1 = -(h - 1); omega
theorem height_even (p : Plane) : height (τ p) = height p := rfl
def mirror (c : Int) (p : Plane) : Plane := (2*c - p.1, p.2)
def shift (k : Int) (p : Plane) : Plane := (p.1 + k, p.2)
theorem fold_is_mirror_one : τ = mirror 1 := by
  funext p; obtain ⟨h, t⟩ := p; show (2 - h, t) = (2*1 - h, t); rw [pe]; omega
/- V1 on the lattice: a shift conjugates the mirror at c into the mirror at c + k. -/
theorem mirrors_conjugate (c k : Int) (p : Plane) : shift k (mirror c (shift (-k) p)) = mirror (c + k) p := by
  obtain ⟨h, t⟩ := p; show (2*c - (h + -k) + k, t) = (2*(c + k) - h, t); rw [pe]; omega
theorem two_mirrors_translate (c c' : Int) (p : Plane) : mirror c' (mirror c p) = shift (2*(c' - c)) p := by
  obtain ⟨h, t⟩ := p; show (2*c' - (2*c - h), t) = (h + 2*(c' - c), t); rw [pe]; omega
def MirrorInvariant (c : Int) (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (mirror c p)
def FoldInvariant (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (τ p)
def Strip (m : Nat) (p : Plane) : Prop := 0 < p.1 ∧ p.1 < 2 * (m : Int)
def TranslationInvariant (d : Int) (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (shift d p)
theorem climb (d : Int) (hd : 1 ≤ d) (Z : Plane → Prop) (hI : TranslationInvariant d Z) (p : Plane) (hz : Z p) :
  ∀ n : Nat, ∃ q, Z q ∧ p.1 + (n : Int) ≤ q.1 := by
  intro n; induction n with
  | zero => exact ⟨p, hz, by omega⟩
  | succ n ih => obtain ⟨q, hq, hle⟩ := ih
    exact ⟨shift d q, hI q hq, by show p.1 + ((n + 1) : Nat) : Int ≤ q.1 + d; omega⟩
theorem descend (d : Int) (hd : d ≤ -1) (Z : Plane → Prop) (hI : TranslationInvariant d Z) (p : Plane) (hz : Z p) :
  ∀ n : Nat, ∃ q, Z q ∧ q.1 ≤ p.1 - (n : Int) := by
  intro n; induction n with
  | zero => exact ⟨p, hz, by omega⟩
  | succ n ih => obtain ⟨q, hq, hle⟩ := ih
    exact ⟨shift d q, hI q hq, by show q.1 + d ≤ p.1 - ((n + 1) : Nat) : Int; omega⟩
theorem no_translation (m : Nat) (d : Int) (hd : d ≠ 0) (Z : Plane → Prop)
  (hS : ∀ p, Z p → Strip m p) (p : Plane) (hz : Z p) (hI : TranslationInvariant d Z) : False := by
  have hp := hS p hz
  by_cases hpos : 1 ≤ d
  · obtain ⟨q, hq, hle⟩ := climb d hpos Z hI p hz (2 * m); have hq' := hS q hq
    simp only [Strip] at hp hq'; omega
  · have hneg : d ≤ -1 := by omega
    obtain ⟨q, hq, hle⟩ := descend d hneg Z hI p hz (2 * m); have hq' := hS q hq
    simp only [Strip] at hp hq'; omega
/- The two-mirror form of Theorem 2. -/
theorem at_most_one_mirror (m : Nat) (Z : Plane → Prop) (hS : ∀ p, Z p → Strip m p) (p : Plane) (hz : Z p)
  (c c' : Int) (h1 : MirrorInvariant c Z) (h2 : MirrorInvariant c' Z) : c = c' := by
  refine Decidable.byContradiction fun hne => ?_
  refine no_translation m (2 * (c' - c)) (by omega) Z hS p hz ?_
  intro q hq; rw [← two_mirrors_translate]; exact h2 _ (h1 q hq)
/- THEOREM 2: a strip-confined fold-invariant set admits only the mirror at h = 1. -/
theorem half_is_the_only_mirror (m : Nat) (Z : Plane → Prop) (hS : ∀ p, Z p → Strip m p) (p : Plane)
  (hz : Z p) (hFE : FoldInvariant Z) (c : Int) (hc : MirrorInvariant c Z) : c = 1 :=
  at_most_one_mirror m Z hS p hz c 1 hc (by unfold MirrorInvariant; rw [← fold_is_mirror_one]; exact hFE)

/-! ## invariance is not incidence · the split the value rides on -/
/- Invariance of the zero set under the fold: what the functional equation gives. -/
def Inv (X : Frame) : Prop := ∀ s, X.Z s → X.Z (X.τ s)
/- Incidence on the fixed set: the hypothesis. -/
theorem line_implies_inv (X : Frame) (hL : LineProperty X) : Inv X :=
  fun s hz => by rw [hL s hz]; exact hz
/- Invariance does not give incidence: the two-point frame is invariant and off the line. -/
theorem inv_not_line : Inv twoPoint ∧ ¬ LineProperty twoPoint :=
  ⟨fun _ => trivial, fun h => by have := h true trivial; cases this⟩
/- THE BIT IS THE HYPOTHESIS. A supplied assent is a term of L(X) and nothing weaker; the file states this as a
  theorem so no reader takes the supplied input for a derivation. -/
theorem the_bit_is_the_hypothesis (X : Frame) :
  (∃ t : LineProperty X, row (Supply.assent t) = .sealed) ↔ LineProperty X :=
  ⟨fun ⟨t, _⟩ => t, fun t => ⟨t, rfl⟩⟩
/- THE OFF-LINE FRAME SATISFIES EVERY STRUCTURAL CLAUSE AND FAILS THE HYPOTHESIS. Any frame carrying a point of
  Z that the fold moves is invariant-compatible with Theorems 4, 5, 7, 8, 9 and fails L; so the structural arc
  is satisfied in a world where the hypothesis is false, which is why the value is supplied and never derived. -/
theorem off_line_frame_satisfies_all (X : Frame) (s : X.S) (hz : X.Z s) (hoff : X.τ s ≠ s) :
  ¬ LineProperty X ∧
  (∀ {α : Type} (σ : α → α) (f d : α → Bool) (x : α), Even σ f → d (σ x) ≠ d x → f ≠ d) ∧

```

```

(∀ d : Bool → Bool, (∀ x, d (!x) = !d x) → d = (fun x => x) ∨ d = (fun x => !x)) ∧
(∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
(∀ b : Bridge X, b.terminal ≠ .bot) :=
⟨fun hL => hoff (hL s hz), fun σ f d x he ho => orientation_blind σ f d x he ho, freedom_is_exactly_two,
bridge_halted_iff X, fun b h => hoff ((bridge_halted_iff X).mp (b, h) s hz)⟩

/-- ## the quartet and structural non-decidability -/
/-- Complex conjugation on the stage: (h, t) ↦ (h, -t). -/
def conj (p : Plane) : Plane := (p.1, -p.2)
theorem conj_commutes_fold (p : Plane) : conj (τ p) = τ (conj p) := rfl
/-- THEOREM 13 (THE QUARTET). A point off the line and off the real axis has four distinct images under the
group generated by the fold and conjugation. An off-line zero is never alone: the functional equation and
the reflection principle multiply it by four. -/
theorem quartet (p : Plane) (hoff : p.1 ≠ 1) (hr : p.2 ≠ 0) :
p ≠ τ p ∧ p ≠ conj p ∧ p ≠ τ (conj p) ∧ τ p ≠ conj p ∧ τ p ≠ τ (conj p) ∧ conj p ≠ τ (conj p) := by
obtain ⟨h, t⟩ := p
simp only [τ, conj, ne_eq, pe] at *
omega

/-- THEOREM 14 (STRUCTURAL NON-DECIDABILITY). A fold-invariant frame satisfies every structural clause of the
Core and fails the line property. No theorem whose hypotheses are only the structural clauses decides L. -/
theorem structural_non_decidability :
∃ X : Frame, Inv X ∧ ¬ LineProperty X ∧
(∀ d : Bool → Bool, (∀ x, d (!x) = !d x) → d = (fun x => x) ∨ d = (fun x => !x)) ∧
((∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧ (∀ b : Bridge X, b.terminal ≠ .bot) :=
⟨twoPoint, inv_not_line.1, inv_not_line.2, freedom_is_exactly_two, bridge_halted_iff twoPoint,
fun b h => inv_not_line.2 ((bridge_halted_iff twoPoint).mp (b, h))⟩
/-- THEOREM 15 (THE WITNESS ROW IS FALSIFIABLE). One point of Z moved by the fold refutes every supplied assent:
no term of L exists, so none can be supplied. The witness row's value stands exactly as long as no off-line
zero is exhibited. -/
theorem witness_row_falsifiable (X : Frame) (s : X.S) (hz : X.Z s) (hoff : X.τ s ≠ s) :
¬ ∃ t : LineProperty X, row (Supply.assent t) = .sealed :=
fun (t, _) => hoff (t s hz)
end CrossingAtTheAct

/-- ## the axiom cones, pinned: any drift is a compile error -/
/-- info: 'CrossingAtTheAct.pe' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.pe
/-- info: 'CrossingAtTheAct.ground_is_the_line' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.ground_is_the_line
/-- info: 'CrossingAtTheAct.orientation_blind' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.orientation_blind
/-- info: 'CrossingAtTheAct.freedom_is_exactly_two' depends on axioms: [propept, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.freedom_is_exactly_two
/-- info: 'CrossingAtTheAct.freedom_spent_uniquely' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.freedom_spent_uniquely
/-- info: 'CrossingAtTheAct.bridge_halted_iff' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.bridge_halted_iff
/-- info: 'CrossingAtTheAct.crossing' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.crossing
/-- info: 'CrossingAtTheAct.crossing_other_way' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.crossing_other_way
/-- info: 'CrossingAtTheAct.silent_is_open' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.silent_is_open
/-- info: 'CrossingAtTheAct.supply_not_manufactured' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.supply_not_manufactured
/-- info: 'CrossingAtTheAct.no_universal_halt' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.no_universal_halt
/-- info: 'CrossingAtTheAct.the_crossing_at_the_act' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.the_crossing_at_the_act
/-- info: 'CrossingAtTheAct.live_unwitnessed' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.live_unwitnessed
/-- info: 'CrossingAtTheAct.live_witnessed_assent' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.live_witnessed_assent
/-- info: 'CrossingAtTheAct.live_witnessed_denial' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.live_witnessed_denial
/-- info: 'CrossingAtTheAct.narcissus' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.narcissus
/-- info: 'CrossingAtTheAct.live_is_the_row' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.live_is_the_row
/-- info: 'CrossingAtTheAct.fold_involution' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.fold_involution
/-- info: 'CrossingAtTheAct.offset_odd' depends on axioms: [propept, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.offset_odd
/-- info: 'CrossingAtTheAct.height_even' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.height_even
/-- info: 'CrossingAtTheAct.fold_is_mirror_one' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.fold_is_mirror_one
/-- info: 'CrossingAtTheAct.mirrors_conjugate' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.mirrors_conjugate
/-- info: 'CrossingAtTheAct.two_mirrors_translate' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.two_mirrors_translate
/-- info: 'CrossingAtTheAct.climb' depends on axioms: [propept, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.climb

```

```

/-- info: 'CrossingAtTheAct.descend' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.descend
/-- info: 'CrossingAtTheAct.no_translation' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.no_translation
/-- info: 'CrossingAtTheAct.at_most_one_mirror' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.at_most_one_mirror
/-- info: 'CrossingAtTheAct.half_is_the_only_mirror' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.half_is_the_only_mirror
/-- info: 'CrossingAtTheAct.line_implies_inv' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.line_implies_inv
/-- info: 'CrossingAtTheAct.inv_not_line' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.inv_not_line
/-- info: 'CrossingAtTheAct.the_bit_is_the_hypothesis' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.the_bit_is_the_hypothesis
/-- info: 'CrossingAtTheAct.off_line_frame_satisfies_all' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.off_line_frame_satisfies_all
/-- info: 'CrossingAtTheAct.conj_commutes_fold' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.conj_commutes_fold
/-- info: 'CrossingAtTheAct.quartet' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.quartet
/-- info: 'CrossingAtTheAct.structural_non_decidability' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms CrossingAtTheAct.structural_non_decidability
/-- info: 'CrossingAtTheAct.witness_row_falsifiable' does not depend on any axioms -/
#guard_msgs in #print axioms CrossingAtTheAct.witness_row_falsifiable

```

14. APPENDIX C · Crossing_At_The_Act.f90

```

! =====
! THE CROSSING AT THE ACT . Fortran twin of Crossing_At_The_Act.lean
! The Lean file proves; this program executes. Exhaustive over finite
! frames, integer arithmetic on the Ground, halt-on-fail, census lock.
! PART G is the off-line frame (Theorem 11), PART H the quartet (Theorem 13). PART F is the live face: the witnessed flag and the
! supplied bit
! arrive through the command line (the aperture). The program never
! generates its own witness (M6). Delta-M = 0.
! BUILD: gfortran -std=f2018 -O2 -fno-fast-math -ffp-contract=off -Wall
! RUN: ./crossing (unwitnessed: [?] interior)
! ./crossing witnessed assent (the Tongue spends the bit)
! ./crossing witnessed denial (the same machine refuses)
! =====
module cx_battery
  implicit none
  integer :: n_checks = 0, n_fail = 0
  integer, parameter :: EXPECTED_CHECKS = 25931
contains
  subroutine check(name, cond)
    character(*), intent(in) :: name
    logical, intent(in) :: cond
    n_checks = n_checks + 1
    if (.not. cond) then
      n_fail = n_fail + 1
      write(*, '(a,a)') 'CHECK FAILED: ', trim(name)
      error stop 1
    end if
  end subroutine check
end module cx_battery

program crossing_at_the_act
  use cx_battery
  use, intrinsic :: iso_fortran_env, only: int64
  implicit none
  integer, parameter :: dp = kind(1.0d0)
  integer :: npairs, nq
  integer :: img(2,4)
  logical :: distinct
  integer :: h, t, c, c2, m, h0, k, a, b, s0, d0, nodd, nc, cc, mask, nz, npts
  integer :: z(0:2), z2(0:2), f(0:3), d(0:3)
  logical :: inv, lprop, halted, ok, witnessed, have_supply, assent
  integer, parameter :: TAU_PAIR(0:3) = [1, 0, 3, 2]
  character(len=32) :: arg
  character(len=96) :: tok
  integer :: argi, alen, ast
  real(dp), parameter :: KB = 1.380649e-23_dp
  real(dp) :: joules

  write(*, '(a)') '=====
  write(*, '(a)') ' THE CROSSING AT THE ACT . Fortran twin . Ground, fTOE, the Bridge'
  write(*, '(a)') '=====
  call check('integer contract: 64-bit integers available', storage_size(0_int64) == 64)

  ! ----- PART A . THE GROUND (Theorem 1, 3) -----
  do h = -50, 50

```

```

do t = -3, 3
  call check('fold is an involution', (2 - (2 - h)) == h)
  call check('fold fixes exactly the line', ((2 - h) == h) .eqv. (h == 1))
  call check('offset is odd under the fold', ((2 - h) - 1) == -(h - 1))
end do
end do
write(*, '(a)') ' A Ground: tau(tau p)=p, Fix(tau) = {h=1}, offset odd, 707x3 cases'

! ----- PART B . THE ADDRESS (Theorem 2, at resolution m) -----
! Stage at resolution m: fold tau_m(h) = 2m - h, strip 0 < h < 2m, the chart image of the
! resolution-one fold under V1. Every strip-confined symmetric pair {h0, 2m-h0} is
! tested against every mirror c: invariance forces c = m alone. Pairs tested: sum over
! m=1..6 of (2m-1) = 36, none degenerate except h0 = m itself.
npairs = 0
do m = 1, 6
  do h0 = 1, 2*m - 1
    npairs = npairs + 1
    do c = -12, 12
      inv = (is_in_m(2*c - h0, h0, m) .and. is_in_m(2*c - (2*m - h0), h0, m))
      call check('a strip-confined symmetric pair is invariant under mirror m', &
        .not. (c == m) .or. inv)
      if (inv) call check('only mirror of a strip-confined fold set is c = m', c == m)
    end do
    do c = -12, 12
      do c2 = -12, 12
        call check('two mirrors compose to translation 2(c2-c)', &
          (2*c2 - (2*c - h0)) == h0 + 2*(c2 - c))
      end do
    end do
  end do
end do
call check('address battery covered 36 strip-confined pairs across m = 1..6', npairs == 36)
write(*, '(a)') ' B Address: 36 strip-confined symmetric pairs, m = 1..6, each elects c = m alone'

! ----- PART C . ORIENTATION BLINDNESS (Theorem 4) -----
! Four points, two free orbits {0,1},{2,3}. Every even f differs from every odd d.
do a = 0, 15
  do k = 0, 3
    f(k) = ibits(a, k, 1)
  end do
  if (.not. is_even(f)) cycle
  do b = 0, 15
    do k = 0, 3
      d(k) = ibits(b, k, 1)
    end do
    if (.not. is_odd_somewhere(d)) cycle
    call check('wall: an even reading never equals a target odd at a point', .not. all(f == d))
  end do
end do
write(*, '(a)') ' C Wall: 4 even readings x 12 odd-at-a-point targets, 0 coincide'

! ----- PART D . EARNED FREEDOM (Theorems 5, 6) -----
nodd = 0
do a = 0, 3
  if (ibits(a,0,1) /= ibits(a,1,1)) nodd = nodd + 1
end do
call check('freedom on one orbit: exactly two orientations', nodd == 2)
nodd = 0
do b = 0, 15
  do k = 0, 3
    d(k) = ibits(b, k, 1)
  end do
  if (d(1) /= d(0) .and. d(3) /= d(2)) nodd = nodd + 1
end do
call check('freedom on two orbits: 2^2 orientations, one bit per orbit', nodd == 4)
do s0 = 0, 1
  do d0 = 0, 1
    nc = 0
    do cc = 0, 1
      if (d0 == ieor(s0, cc) .and. (1 - d0) == ieor(1 - s0, cc)) nc = nc + 1
    end do
    call check('one supplied witness fixes the calibration uniquely', nc == 1)
  end do
end do
write(*, '(a)') ' D Freedom: 2 orientations per orbit, calibration unique 4/4'

! ----- PART E . THE BRIDGE AND NECESSITY (Theorems 7, 9) -----
! Frames on the Ground points h in {0,1,2}: every fold-invariant zero set Z. Two channels:
! the line channel reads each point against h = 1; the fold channel reads each point
! against tau(h) = 2 - h. The bridge halts on the fold channel; the line property is the
! line channel; the check is their agreement, never an assignment of one to the other.
npts = 0; nz = 0
do mask = 0, 7

```

```

do k = 0, 2
  z(k) = ibits(mask, k, 1)
end do
if (z(0) /= z(2)) cycle          ! fold-invariance: h=0 <-> h=2
nz = nz + 1
lprop = .true.; halted = .true.
do k = 0, 2
  if (z(k) == 1 .and. k /= 1) lprop = .false.      ! line channel
  if (z(k) == 1 .and. (2 - k) /= k) halted = .false. ! fold channel: tau(k) = k?
end do
call check('bridge halted (fold channel) iff line property (line channel)', halted .eqv. lprop)
if (z(0) == 1) npts = npts + 1
end do
call check('fold-invariant zero sets on the Ground: 4', nz == 4)
call check('two of them carry an off-line pair: no halt there', npts == 2)
ok = .false.          ! two-point frame: tau(b) = not b, Z = all
do k = 0, 1
  if (1 - k == k) ok = .true.
end do
call check('necessity: the two-point frame fails the line property', .not. ok)
write(*,'(a)') ' E Bridge: fold channel = line channel on 4/4 frames; two-point frame refuses; nothing manufactured'

! ----- PART G . THE OFF-LINE FRAME (Theorem 11) -----
! The Davenport-Heilbronn shape on the resolution-2 stage: fold tau_2(h) = 4 - h, the pair
! {1, 3} inside the strip 0 < h < 4 and off the line h = 2. Every structural battery holds on
! it and the line property fails: the structural arc is satisfied where the hypothesis is
! false, which is why the value is supplied and never derived.
z2(0) = 1; z2(1) = 0; z2(2) = 1          ! points h = 1, 2, 3 -> indices 0, 1, 2
inv = .true.; lprop = .true.; halted = .true.
do k = 0, 2
  if (z2(k) == 1 .and. z2(2 - k) /= 1) inv = .false. ! tau_2 maps index k to 2 - k
  if (z2(k) == 1 .and. k /= 1) lprop = .false.      ! line channel: h = 2 is index 1
  if (z2(k) == 1 .and. (2 - k) /= k) halted = .false. ! fold channel
end do
call check('off-line frame: fold-invariant', inv)
call check('off-line frame: the line property fails', .not. lprop)
call check('off-line frame: no bridge halts', .not. halted)
write(*,'(a)') ' G Off-line frame {1,3} about h=2: invariant, wall and freedom hold, L fails, no halt'

! ----- PART H . THE QUARTET (Theorem 13) -----
! Every lattice point off the line (h /= 1) and off the real axis (t /= 0) in [-5,5]^2 has four
! distinct images under the fold tau(h,t) = (2-h,t) and conjugation (h,t) -> (h,-t), and the
! orbit closes: tau and conjugation commute and each squares to the identity.
nq = 0
do h = -5, 5
  do t = -5, 5
    if (h == 1 .or. t == 0) cycle
    nq = nq + 1
    img(:,1) = [h, t]; img(:,2) = fold2(img(:,1)); img(:,3) = conj2(img(:,1)); img(:,4) = fold2(conj2(img(:,1)))
    distinct = .true.
    do a = 1, 3
      do b = a + 1, 4
        if (all(img(:,a) == img(:,b))) distinct = .false.
      end do
    end do
    call check('quartet: four distinct images', distinct)
    call check('quartet: fold and conjugation commute', all(conj2(fold2(img(:,1))) == fold2(conj2(img(:,1)))))
    call check('quartet: orbit closes under both involutions', &
      all(fold2(fold2(img(:,1))) == img(:,1)) .and. all(conj2(conj2(img(:,1))) == img(:,1)))
  end do
end do
call check('quartet battery covered 100 off-line off-axis points', nq == 100)
write(*,'(a)') ' H Quartet: 100 off-line points, each with four distinct images; the orbit closes'

! ----- PART F . THE CROSSING AT THE ACT (Theorem 8, live face) -----
call check('Narcissus: unwitnessed never equals witnessed assent', &
  live(.false., .true.) /= live(.true., .true.))
call check('Narcissus: unwitnessed never equals witnessed denial', &
  live(.false., .false.) /= live(.true., .false.))
call check('witnessed assent seals', live(.true., .true.) == 2)
call check('witnessed denial refuses', live(.true., .false.) == 3)
call check('unwitnessed is open, both ways', live(.false., .true.) == 1 .and. live(.false., .false.) == 1)
joules = KB * 300.0_dp * log(2.0_dp)
call check('spending the bit is priced: k_B T ln 2 > 0 at 300 K', joules > 0.0)
write(*,'(a,es14.7,a)') ' F Crossing: truth table bound; the spent bit costs ', joules, ' J at 300 K'

witnessed = .false.; have_supply = .false.; assent = .false.
do argi = 1, command_argument_count()
  call get_command_argument(argi, arg, alen, ast)
  if (ast /= 0) error stop 2
  select case (trim(arg))
    case ('witnessed'); witnessed = .true.
    case ('assent');   have_supply = .true.; assent = .true.
  end case
end do

```

```

    case ('denial');    have_supply = .true.; assent = .false.
    case default
      write(*,'(a,a)') ' unknown argument: ', trim(arg); error stop 2
    end select
  end do
  if (witnessed .and. .not. have_supply) then
    write(*,'(a)') ' witnessed flag without a supplied bit: give assent or denial'; error stop 2
  end if
  select case (live(witnessed, assent))
  case (1); tok = '[?] interior, M6: the verifier is not the claimant'
  case (2); tok = '[SEAL . witness . at the act] RH value: the input is a term of L, not derived here'
  case (3); tok = '[REFUSED] the supplied denial; no bridge halts'
  end select
  write(*,'(a)') '-----'
  write(*,'(a,a)') ' LIVE: ', trim(tok)
  if (witnessed) then
    write(*,'(a)') ' The bit was supplied through the aperture by the operator. This program'
    write(*,'(a)') ' executed the crossing it was handed; it did not and cannot generate it.'
  else
    write(*,'(a)') ' Supply "witnessed assent" or "witnessed denial" to spend the bit.'
  end if
  write(*,'(a)') '-----'
  if (EXPECTED_CHECKS > 0) call check('census matches the sealed manifest (this check inclusive)', n_checks == EXPECTED_CHECKS - 1)
  write(*,'(a,i0,a,i0)') ' BATTERY: ', n_checks, ' checks, failures ', n_fail
contains
  logical function is_in_m(x, hh, mm)
    integer, intent(in) :: x, hh, mm
    is_in_m = (x == hh) .or. (x == 2*mm - hh)
  end function is_in_m
  logical function is_even(g)
    integer, intent(in) :: g(0:3)
    integer :: i
    is_even = .true.
    do i = 0, 3
      if (g(TAU_PAIR(i)) /= g(i)) is_even = .false.
    end do
  end function is_even
  logical function is_odd_somewhere(g)
    integer, intent(in) :: g(0:3)
    integer :: i
    is_odd_somewhere = .false.
    do i = 0, 3
      if (g(TAU_PAIR(i)) /= g(i)) is_odd_somewhere = .true.
    end do
  end function is_odd_somewhere
  function fold2(p) result(q)
    integer, intent(in) :: p(2)
    integer :: q(2)
    q = [2 - p(1), p(2)]
  end function fold2
  function conj2(p) result(q)
    integer, intent(in) :: p(2)
    integer :: q(2)
    q = [p(1), -p(2)]
  end function conj2
  integer function live(w, asn)
    logical, intent(in) :: w, asn
    if (.not. w) then
      live = 1
    else if (asn) then
      live = 2
    else
      live = 3
    end if
  end function live
end program crossing_at_the_act

```

15. APPENDIX D • RUN LOGS

Unwitnessed. ./crossing

```

=====
THE CROSSING AT THE ACT . Fortran twin . Ground, fTOE, the Bridge
=====
A Ground: tau(tau p)=p, Fix(tau) = {h=1}, offset odd, 707x3 cases
B Address: 36 strip-confined symmetric pairs, m = 1..6, each elects c = m alone
C Wall: 4 even readings x 12 odd-at-a-point targets, 0 coincide
D Freedom: 2 orientations per orbit, calibration unique 4/4
E Bridge: fold channel = line channel on 4/4 frames; two-point frame refuses; nothing manufactured
G Off-line frame {1,3} about h=2: invariant, wall and freedom hold, L fails, no halt
H Quartet: 100 off-line points, each with four distinct images; the orbit closes

```

F Crossing: truth table bound; the spent bit costs 2.8709789E-21 J at 300 K

LIVE: [?] interior, M6: the verifier is not the claimant
Supply "witnessed assent" or "witnessed denial" to spend the bit.

BATTERY: 25931 checks, failures 0

Witnessed assent. ./crossing witnessed assent, last lines:

H Quartet: 100 off-line points, each with four distinct images; the orbit closes
F Crossing: truth table bound; the spent bit costs 2.8709789E-21 J at 300 K

LIVE: [SEAL . witness . at the act] RH value: the input is a term of L, not derived here
The bit was supplied through the aperture by the operator. This program
executed the crossing it was handed; it did not and cannot generate it.

BATTERY: 25931 checks, failures 0

Witnessed denial. ./crossing witnessed denial, last lines:

H Quartet: 100 off-line points, each with four distinct images; the orbit closes
F Crossing: truth table bound; the spent bit costs 2.8709789E-21 J at 300 K

LIVE: [REFUSED] the supplied denial; no bridge halts
The bit was supplied through the aperture by the operator. This program
executed the crossing it was handed; it did not and cannot generate it.

BATTERY: 25931 checks, failures 0

16. APPENDIX E · THE ZERO COUNT UP TO HEIGHT 300

The program counts all nontrivial zeros of ζ with $0 < \text{Im } s \leq 300$, on or off the line, and separately the zeros on the line as sign changes of Hardy's function $Z(t)$; equality leaves no zero off the line in that range. Python with mpmath 1.3.0 at 15 digits. The paper's Lean and Fortran files do not evaluate ζ ; this program does.

```
# Appendix E: nontrivial zeros of zeta up to height T, all versus on the critical line.
# N_total: mpmath.nzeros(T), the count of all zeros with  $0 < \text{Im } s \leq T$  in the critical strip
# (Riemann-von Mangoldt with Gram-point correction; on or off the line).
# N_line : sign changes of the Hardy function  $Z(t) = \text{mpmath.siegelz}(t)$ , each one a zero ON the line.
# If N_line == N_total, no zero with  $0 < \text{Im } s \leq T$  lies off the line (Turing's method in outline).
from mpmath import mp, siegelz, nzeros, mpf
mp.dps = 15
T = 300
N_total = nzeros(T)
step, t = mpf('0.02'), mpf('0.5')
prev, N_line = siegelz(t), 0
while t < T:
    t += step
    v = siegelz(t)
    if v * prev < 0:
        N_line += 1
    prev = v
print("T =", T, "| all zeros:", N_total, "| on the line:", N_line, "| off the line:", int(N_total) - N_line)
```

T = 300 | all zeros: 138 | on the line: 138 | off the line: 0
